

파이토치 임베딩(Embedding)

1. 전처리: 기계의 언어로 번역하기 전 '청소'

컴퓨터는 "사과"와 "사과!"를 다른 단어로 인식함. 단어 수를 줄여야 효율이 올라가기 때문에, 먼저 대문자를 소문자로 바꾸고 구두점을 떼어내는 '세탁' 과정이 필요함.

텍스트 청소하기 (clean_words)

이 함수는 문장을 받아서 소문자로 밀고, 마침표(.)나 물음표(?) 같은 군더더기를 제거함.

```
def clean_words(input_str):
    punctuation = '.,;:"!?"'"_-' # 제거할 구두점 목록

    # 1. 소문자로 바꾸고 줄바꿈 제거 후 공백 단위로 쪼개기
    word_list = input_str.lower().replace('\n', ' ').split()

    # 2. 단어 양옆에 붙은 구두점 찌꺼기들 제거
    word_list = [word.strip(punctuation) for word in word_list]
    return word_list

line = "Impossible, Mr. Bennet, impossible, when I am not acquainted with him"
words_in_line = clean_words(line)

# 결과: ['impossible', 'mr', 'bennet', 'impossible', 'when', 'i', 'am', 'not',
'acquainted', 'with', 'him']
```

2. 단어 사전과 인덱스: 단어에 '학번' 매기기

컴퓨터는 글자 자체를 못 다루니까, 우리만의 '사전'을 만들어서 각 단어에 0번부터 번호를 붙여줘야 함. 이걸 인덱싱(Indexing)이라고 함.

사전 만들기

```
# 중복을 제거하고 알파벳 순서대로 정렬해서 리스트 생성
word_list = sorted(set(clean_words(text)))

# { '단어': 인덱스번호 } 형태의 딕셔너리 만들기
word2index_dict = {word: i for (i, word) in enumerate(word_list)}

# 예: 전체 7,261개 단어 중 'impossible'은 3,394번이라는 번호표를 받음
print(len(word2index_dict), word2index_dict['impossible'])
```

3. 원핫 인코딩 (One-hot Encoding): 무식하지만 확실한 방법

단어를 숫자로 바꾸는 가장 기초적인 방법임. 사전 크기만큼 긴 칸을 만들고, 자기 번호 칸에만 '1'(전등 켜기)을 넣고 나머지는 0으로 채우는 방식임.

📖 파이토치 원핫 텐서 출력 예시

```
import torch

# 문장 길이(11단어) x 사전 크기(7261개)의 0으로 가득 찬 판을 만들
word_t = torch.zeros(len(words_in_line), len(word2index_dict))

for i, word in enumerate(words_in_line):
    word_index = word2index_dict[word]
    word_t[i][word_index] = 1 # 자기 번호 자리만 1로 바꿈
    print(f"{i}번 단어: {word} (인덱스 {word_index})")

print(word_t.shape) # 결과: torch.Size([11, 7261])
```

⚠ 원핫 인코딩의 한계 (왜 임베딩을 써야 하나?)

1. **공간 낭비**: 단어 하나 표현하려고 숫자 7,261개를 쓰는 건 너무 비효율적임. (대부분이 0인 '희소 벡터' 문제)
2. **관계 파악 불가**: "사과"와 "배"는 비슷한 과일이지만, 원핫 벡터로 보면 두 단어의 연관성을 전혀 찾을 수 없음. 그냥 서로 다른 위치의 전등일 뿐임.

4. 임베딩 (Embedding): 의미를 담은 '지도 좌표'

임베딩은 단어를 0과 1이 아닌, **실수(소수점 숫자)**를 사용해 훨씬 짧은 벡터로 표현함. 비슷한 맥락에서 쓰이는 단어들을 공간상에서 가깝게 배치하는 것이 핵심임.

🍎 이해하기 쉬운 수동 임베딩 예시

우리가 직접 '종류'와 '색상'이라는 두 개의 축(차원)으로 지도를 만든다고 가정해봄.

- **X축 (종류)**: 과일은 0.1 근처, 강아지는 0.8 근처
- **Y축 (색상)**: 빨간색은 0.1 근처, 노란색은 0.5 근처

단어	X좌표 (종류)	Y좌표 (색상)	의미
사과	0.1	0.1	과일이면서 빨간색임
레몬	0.1	0.5	과일이면서 노란색임
골든 리트리버	0.8	0.5	강아지이면서 노란색임

이렇게 하면 "사과"와 "레몬"은 과일 축($X=0.1$)이 같아서 가깝게 위치하게 됨. 이게 바로 임베딩의 힘임!

🧠 임베딩의 마법: 벡터 연산

잘 학습된 임베딩 공간에서는 이런 계산도 가능해짐.

사과 - 빨강 + 노랑 = 레몬 (빨간 과일에서 빨강을 빼고 노랑을 더하면 노란 과일이 됨)

5. 파이토치 실전: nn.Embedding

실제 딥러닝에서는 사람이 좌표를 일일이 찍지 않음. 신경망이 수천만 개의 문장을 읽으면서 스스로 단어 사이의 관계를 파악해 최적의 좌표를 찾아냄.

```
import torch.nn as nn

# 1. 7261개의 단어를 각각 100개의 숫자로 표현하겠다고 설정
# (원핫 인코딩의 7261차원을 100차원으로 확 줄이는 것임)
embedding_layer = nn.Embedding(7261, 100)

# 2. 'impossible' (3394번)을 입력하면 100개의 실수가 담긴 벡터가 튀어나옴
input_idx = torch.tensor([3394])
output = embedding_layer(input_idx)

print(output.shape) # 결과: torch.Size([1, 100])
```

6. 최종 요약: 원핫 인코딩 vs 임베딩 비교

비교 항목	원핫 인코딩 (One-hot)	임베딩 (Embedding)
표현 방식	0과 1로만 구성 (희소 벡터)	소수점 숫자로 구성 (밀집 벡터)
벡터의 길이	전체 사전 크기 (엄청 길고 비효율적)	사용자가 정한 크기 (보통 100~1000)
단어 간 관계	전혀 알 수 없음 (거리 다 똑같음)	비슷한 의미는 가깝게 위치함
학습 가능 여부	고정되어 있음 (학습 안 함)	모델이 스스로 최적의 값을 학습함
비유	이름표 번호 순서대로 나열하기	의미 지도 위에 위도와 경도로 좌표 찍기